

# AI Critique

---

AI accelerated the mechanical parts of this assignment, especially converting repeated manual cases into typed, data-driven Playwright tests and producing a nine-cell browser runner. However, its first proposals were incomplete in ways that could have produced misleading green reports. It assumed visible labels were accessible labels, so `getByLabel` waited forever on markup that lacked proper associations. It also tried to fill alphabetic text into a number input, an action Playwright intentionally rejects. More seriously, the initial Coupon Admin oracle checked native form validity and immediately asserted that no row existed. React rendered the invalid coupon slightly later, allowing Chromium to pass falsely while Firefox and WebKit exposed the race. Automatic failure screenshots then ran after cleanup and hid the offending row.

These mistakes occurred because the model inferred normal semantic HTML, synchronous state updates, and conventional validation from generic Playwright patterns. It did not initially account for this SUT's inaccessible labels, asynchronous rendering, or the difference between browser constraint validation and server-side business validation. A prompt that only requested "working tests" would have reinforced the wrong goal: making tests pass instead of proving the requirement.

The main principle I learned is to treat AI output as a hypothesis, not evidence. Every locator must be checked against the actual accessibility tree, every oracle must come from the requirement rather than the current implementation, and every asynchronous negative assertion needs a trustworthy settling condition. Cross-browser disagreement is diagnostic information, not noise. Human review must also inspect screenshots and raw artifacts; otherwise cleanup, timing, or a weak assertion can make a genuine defect disappear from the report.